

VERİ TABANI YÖNETİM SİSTEMLERİ PROJE RAPORU

VERİ TABANI YÖNETİM SİSTEMLERİ PROJE RAPORU

Proje Adı: BranchSync: Merkezi Denetimli Şube Stok ve Envanter Yönetim Sistemi

Ders Sorumlusu: Dr. Öğr. Üyesi Mehmet Ü. AK

1. Proje Tanımı ve Amacı

1.1. Projenin Tanımı

Bu proje; günümüz modern perakende, teknoloji ve lojistik sektörlerinde faaliyet gösteren franchise veya zincir mağaza modellerinin en büyük yapısal problemlerinden biri olan 'merkezi envanter ve dağıtık stok yönetimini' çözmek amacıyla tasarlanmıştır. 'BranchSync: Merkezi Denetimli Şube Stok ve Envanter Yönetim Sistemi' tabanlı bir ilişkisel veri tabanı tasarımıdır. Sistem; bir merkeze bağlı olarak çalışan farklı konumlardaki şubelerin anlık ürün stok durumlarını, bu şubelerde gerçekleştirilen mali satış işlemlerini ve şubelerin kendi aralarında gerçekleştirdikleri lojistik ürün transfer süreçlerini tek bir güvenli merkezden (PostgreSQL Veri Tabanı Yönetim Sistemi) takip etmeyi ve denetlemeyi sağlar.

1.2. Projenin Amacı ve Hedefleri

Projenin temel amacı, veri tabanı yönetim sistemleri prensiplerine (ACID kuralları, ilişkisel bütünlük ve normalizasyon formları) tam uyumlu bir mimari inşa ederek şu operasyonel hedeflere ulaşmaktır:

Veri Bütünlüğü ve Güvenliği: Sisteme girilen stok miktarlarını ve finansal satış tutarlarının iş kuralları kısıtlamaları (CHECK Constraints) sayesinde asla negatif veya hatalı değerler almamalarını garanti etmek.

Mükerrer Kayıt Engelleme: Bir şubede yer alan belirli bir ürünün stok kaydının benzersizliğini koruyarak (UNIQUE Constraints), ilişkisel köprü tablolar üzerinde veri tekrarını (redundancy) sifra indirmek.

Lojistik ve Tedarik Optimizasyonu: Şubeler arası ürün transfer mekanizmasını veri tabanı seviyesinde kayıt altına alarak, bir şubenin kendi kendine transfer yapmasını engellemek ve şubeler arası ürün akışını şeffaf hale getirmek.

Karar Destek ve Raporlama Yeteneği: Gelişmiş ilişkisel sorgular (JOIN), gruplamalar (GROUP BY) ve iç içe geçmiş alt sorgular (Subquery) kullanarak; en çok ciro yapan şubeleri listelemek, anlık olarak kritik stok seviyesinin altına düşen ürünleri tespit etmek ve işletme yönetiminin hızlı kararlar almasını sağlayacak finansal analiz raporları üretmek.

2. Proje Gereksinimleri ve Kurallar

Sistemin mantıksal ve veri tabanı seviyesinde zorunlu kılınan kurallar aşağıda listelenmiştir:

Sistemde toplam 5 temel tablo bulunmaktadır (urunler, subeler, sube_stok, satislar, stok_transferleri).

Her tablonun benzersiz bir Primary Key (PK) alanı vardır ve tablolar birbirine Foreign Key (FK) bağları ile sıkı bir şekilde ilişkilendirilmiştir.

Bir şubede envanter miktarı belirlenen kritik seviyenin altına düştüğünde sistem bunu dinamik olarak analiz edebilir.

Stok miktarlar ve satış tutarları veri tabanı kısıtlamaları (CHECK) sayesinde sıfırın altına düşemez.

Aynı şubeye aynı ürün için mükerrer bir stok satışı açılması engellenmiştir (Composite Unique kısıtlaması).

3. Varlık-Bağıntı (ER) Diyagramı

4. Veri Tabanı Tablo Yapıları ve Kodları (DDL)

Aşağıdaki SQL betiği, veri tabanındaki 5 ana tablonun ilişkisel bütünlük ve kısıtlama kurallarına uygun olarak PostgreSQL üzerinde oluşturulmasını sağlar:

```
-- 1. Ürünlerin ana bilgilerini tutan tablo
CREATE TABLE urunler (
    urun_id SERIAL PRIMARY KEY,
    barkod VARCHAR(50) UNIQUE NOT NULL,
    urun_adi VARCHAR(100) NOT NULL,
    kategori VARCHAR(50),
    kritik_seviye INTEGER DEFAULT 10,
    CONSTRAINT chk_kritik_seviye CHECK (kritik_seviye >= 0)
);

-- 2. Şubelerin bilgilerini tutan tablo
CREATE TABLE subeler (
    sube_id SERIAL PRIMARY KEY,
    sube_adi VARCHAR(100) NOT NULL,
    sehir VARCHAR(50) NOT NULL
);

-- 3. Şube anlamlı stok durum tablosu (Çoktan Çoğa - M:N İlişki Köprüsü)
CREATE TABLE sube_stok (
    stok_id SERIAL PRIMARY KEY,
    sube_id INTEGER REFERENCES subeler(sube_id) ON DELETE CASCADE,
    urun_id INTEGER REFERENCES urunler(urun_id) ON DELETE CASCADE,
    miktar INTEGER DEFAULT 0,
    CONSTRAINT chk_stok_miktar CHECK (miktar >= 0),
    CONSTRAINT uq_sube_urun UNIQUE (sube_id, urun_id)
);

-- 4. Şubelerde yapılan satışların kayıtları
CREATE TABLE satislar (
    satis_id SERIAL PRIMARY KEY,
    sube_id INTEGER REFERENCES subeler(sube_id) ON DELETE RESTRICT,
    islem_tarihi TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    toplam_tutar NUMERIC(10, 2) NOT NULL,
    CONSTRAINT chk_toplam_tutar CHECK (toplam_tutar > 0)
```

```
);
-- 5. Subeler arasındaki yapılan stok transfer hareketleri tablosu
CREATE TABLE stok_transferleri (
transfer_id SERIAL PRIMARY KEY,
kaynak_sube_id INTEGER REFERENCES subeler(sube_id) ON DELETE RESTRICT,
hedef_sube_id INTEGER REFERENCES subeler(sube_id) ON DELETE RESTRICT,
urun_id INTEGER REFERENCES urunler(urun_id) ON DELETE RESTRICT,
miktar INTEGER NOT NULL,
transfer_tarihi TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT chk_transfer_miktar CHECK (miktar > 0),
CONSTRAINT chk_farkli_sube CHECK (kaynak_sube_id != hedef_sube_id)
);
```

5. Test Verilerinin Yüklmesi (INSERT)

Tasarlanan veri yapısının test etmek, ilişkileri doğrulamak ve analiz sorgularına anlamlı çıktılar üretmek amacıyla kullanılan örnek SQL veri seti:

```
-- Ürün Tanımlamaları
INSERT INTO urunler (barkod, urun_adi, kategori, kritik_seviye) VALUES
('8690001', 'Logitech G305 Kablosuz Mouse', 'Çevre Birimleri', 15),
('8690002', 'Asus TUF Gaming Monitör 24"', 'Monitör', 5),
('8690003', 'Samsung Galaxy S23 128GB', 'Telefon', 8),
('8690004', 'Lenovo IdeaPad Gaming 3', 'Bilgisayar', 3),
('8690005', 'Kingston 16GB DDR4 RAM', 'Bilgisayar Bileşenleri', 20),
('8690006', 'Philips Hue Akıllı Ampul', 'Akıllı Ev', 12);

-- Şube Tanımlamaları
INSERT INTO subeler (sube_adi, sehir) VALUES
('MAKÜ Kampüs Merkez Şube', 'Burdur'),
('Bucak Franchise Şube', 'Burdur'),
('Göhlhisar Şube', 'Burdur');

-- Şubelerin Mevcut Stok Girişleri
INSERT INTO sube_stok (sube_id, urun_id, miktar) VALUES
(1, 1, 50), (1, 2, 4), (1, 3, 12), (1, 4, 8), (1, 5, 35),
(2, 1, 20), (2, 2, 8), (2, 3, 2), (2, 4, 1), (2, 5, 15),
(3, 1, 10), (3, 3, 5), (3, 6, 25);

-- Gerçekleşen Satışlar
INSERT INTO satislar (sube_id, islem_tarihi, toplam_tutar) VALUES
(1, '2026-05-10 14:30:00', 12500.00),
(1, '2026-05-11 11:15:00', 32000.00),
(2, '2026-05-12 16:45:00', 2750.00),
(2, '2026-05-13 09:30:00', 31500.00),
(3, '2026-05-14 12:00:00', 1350.00),
(3, '2026-05-15 18:20:00', 26000.00);
```

```
-- Subeler Arası Transfer Kayıtları
INSERT INTO stok_transferleri (kaynak_sube_id, hedef_sube_id, urun_id, miktar,
transfer_tarihi) VALUES
(1, 2, 3, 5, '2026-05-01 10:00:00'),
(1, 3, 1, 10, '2026-05-03 14:00:00'),
(2, 1, 5, 2, '2026-05-05 11:30:00');
```

6. Gelişimi Raporlama ve Analiz Sorguları

Proje tablonunda zorunlu tutulan; JOIN, GROUP BY ve Subquery (Alt Sorgu) gibi ileri düzey SQL tekniklerinin harmanlandığı yönetimsel sorgular:

Rapor 1: Subelerin Toplam Satış Adedi ve Ciro Dağılımı (JOIN & GROUP BY)

Açıklama: Hangi subenin kaç adet satış işlemi gerçekleştirdiğini ve toplamda ne kadarlık bir ciro elde ettiğini listeleterek sube performans analizi sunar.

```
SELECT
s.sube_adi,
s.sehir,
COUNT(sa.satis_id) AS toplam_satis_adedi,
SUM(sa.toplam_tutar) AS toplam_ciro
FROM subeler s
INNER JOIN satislar sa ON s.sube_id = sa.sube_id
GROUP BY s.sube_id, s.sube_adi, s.sehir
ORDER BY toplam_ciro DESC;
```

Rapor 2: Kritik Seviyenin Altına Düşen Ürünlerin Listesi (Subquery)

Açıklama: Subeler bazında, mevcut envanter miktarı o ürünün kendi tablosunda belirlenmiş kritik eşik değerine eşit veya bu değerlerin altına düşmüş olan kritik durumları alt sorgu yardımıyla tespit eder.

```
SELECT
sub.sube_adi,
u.urun_adi,
ss.miktar AS mevcut_stok,
u.kritik_seviye
FROM sube_stok ss
INNER JOIN subeler sub ON ss.sube_id = sub.sube_id
INNER JOIN urunler u ON ss.urun_id = u.urun_id
WHERE ss.miktar <= (
SELECT kritik_seviye
FROM urunler
WHERE urun_id = ss.urun_id
)
ORDER BY ss.miktar ASC;
```

Rapor 3: Genel Ortalama Satışın Üzerindeki Büyük İlemler (Subquery & WHERE)

Açıklama: Sistemdeki tüm subelerde yapılan tüm satış işlemlerinin genel bir mali ortalamasının çıkar ve bu ortalama sepet tutarının üstünde kalan yüksek hacimli satışlar listeler.

```
SELECT
sa.satis_id,
s.sube_adi,
sa.islem_tarihi,
sa.toplam_tutar
FROM satislar sa
INNER JOIN subeler s ON sa.sube_id = s.sube_id
WHERE sa.toplam_tutar > (
SELECT AVG(toplam_tutar)
FROM satislar
)
ORDER BY sa.toplam_tutar DESC;
```